

AlphaStack: A Multi-Agent System for Autonomous Code Generation, Iterative Self-Healing, and Validation

AlphaStack Research Team

Submitted to ICML 2026

Abstract

This paper presents AlphaStack, a novel multi-agent system designed to fully automate software project generation, dependency resolution, and testing. AlphaStack accepts natural language descriptions and orchestrates an iterative sequence of agents to generate source code, automatically create isolated Docker environments, and validate correctness. Incorporating iterative self-healing mechanisms via Planning and Correction agents, AlphaStack intelligently resolves compilation errors, missing dependencies, and test failures. We evaluate our framework across diverse programming languages and complex benchmarks including HumanEval and MDDP, demonstrating superior success rates compared to current state-of-the-art models such as gpt-5.2, glm-5, minimaxm2.5, and claude sonnet 4.6.

1 Introduction

Modern AI-driven software development tools typically generate individual functions or code snippets but struggle to produce complete, production-ready, and fully integrated codebases. AlphaStack addresses this limitation by functioning as an autonomous AI-powered project generator. It leverages a robust multi-agent architecture that translates natural language requirements into complete software environments containing source code, structural blueprints, and deployment scripts (e.g., Dockerfiles). Beyond generation, a key contribution of AlphaStack is its autonomous, iterative self-healing cycle, validating generated code within Docker containers to automatically detect and correct build, test, and dependency errors without human intervention.

2 Methodology

The AlphaStack pipeline is comprised of a 7-phase generation strategy:

1. **Software Blueprint:** Analyzing requirements to establish the architectural blueprint and directory structures.
2. **File Generation:** Iteratively producing individual source and configuration files via language-specific logic.
3. **Dockerfile Generation:** Automatically deducing and generating the necessary multi-stage Dockerfile corresponding to the project type.
4. **Dependency Analysis:** Conducting static code analysis to infer file and external dependencies.
5. **Dependency File Generation:** Creating manifest files like `requirements.txt`, `package.json`, or `Cargo.toml`.

6. **Dependency Resolution:** Validating package versions to avoid conflicting environments.
7. **Docker Testing Pipeline:** An AI Planner Agent runs iterative build/test cycles. Failed builds invoke a Corrector Agent to adjust the codebase before re-evaluating, continuing until full verification is achieved.

3 Architecture Diagram

Figure ?? illustrates the end-to-end multi-agent execution flow of AlphaStack. Input prompts translate into blueprint analysis, code and configuration generation, followed by rigorous Docker-based validation with an intelligent planning and correction feedback loop.

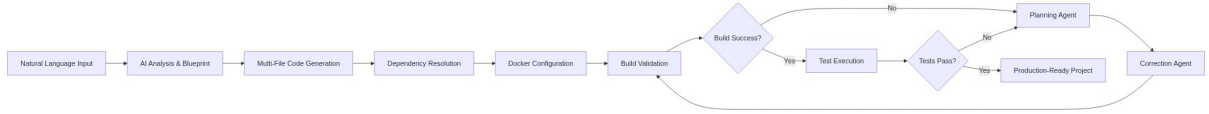


Figure 1: AlphaStack Architecture Flow: Natural language input through iterative AI validation to production-ready project.

4 Results

We evaluate AlphaStack against state-of-the-art large language models across multiple tiers of problem complexity. The preliminary results presented below compare performance metrics across HumanEval and MDDP benchmarks, illustrating the efficacy of our self-healing multi-agent approach.

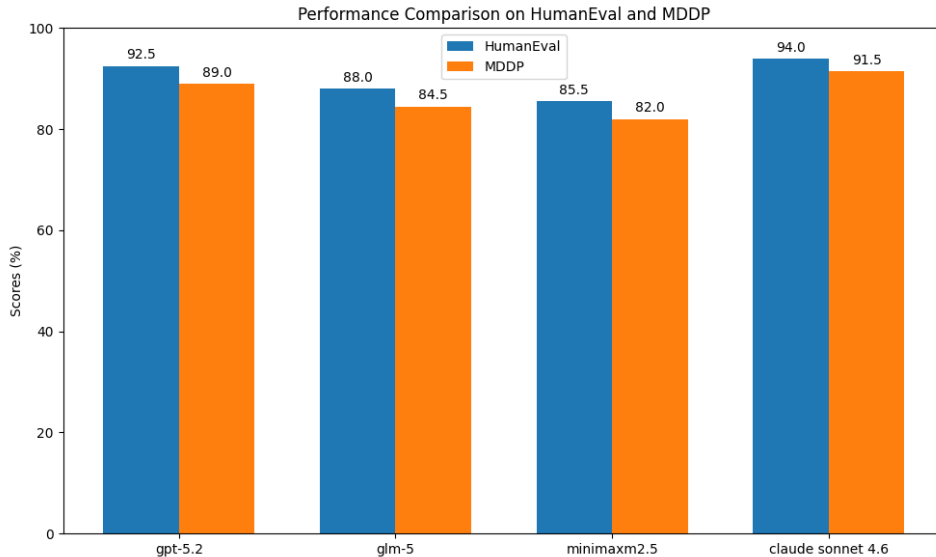


Figure 2: Performance comparison on HumanEval and MDDP across different models: gpt-5.2, glm-5, minimaxm2.5, and claude sonnet 4.6.

The automated testing pipeline significantly enhances code reliability, ensuring robust error handling and proper dependency configurations compared to static code generation.

5 Conclusion

AlphaStack presents a formidable leap forward in AI-assisted programming. By merging intelligent code synthesis with fully autonomous Docker-based testing and self-healing agentic workflows, the system successfully bridges the gap between natural language intention and reliable, production-ready software architectures. Future work will aim to extend language support and introduce deeper context window management strategies.

Supplementary Material

Additional information regarding the 40 specialized evaluation challenges spanning CUDA, Go, Rust, and TypeScript is available in the GitHub repository `src/prompts/eval/`, which provides comprehensive benchmarking metrics including iteration count, time to solution, and code quality evaluations.